

2万件のパイプラインを データ需要の行動ログ として読み解く

約20,000件のODMデータパイプライン（XML）を解析し、繰り返し発生してきたデータ需要と加工パターンを構造として復元する。目的は「2万本を移植すること」ではなく、複数部門・複数用途から再利用できる共通データ資産の候補を、コードの利用実態から発見すること。

AS-IS

同じ中間データが Pipeline ごとに作り直されている

CUSTOMER

ACCOUNT

TRANSACTION

Pipeline A
同じJOINを個別に実行 → 個別加工

Pipeline B
同じJOINを個別に実行 → 個別加工

Pipeline C
同じJOIN + PRODUCT → 個別加工

営業用 OUTPUT

リスク用 OUTPUT

マーケティング OUTPUT

同じ結合結果が Pipeline ごとに別々に生成される。加工の重複・定義のばらつき・計算コストの重複が同時に発生する。

TO-BE

共通中間層を一度だけ作り、後続から再利用する

CUSTOMER

ACCOUNT

TRANSACTION

PRODUCT

共通中間層（1度だけ生成）
CUSTOMER + ACCOUNT
+ TRANSACTION
JOINキー・粒度・定義を標準化

Pipeline A
営業向け個別加工

Pipeline B
リスク向け個別加工

Pipeline C
+ PRODUCT で個別加工

OUTPUT A

OUTPUT B

OUTPUT C

最大の問いは「どこまでを共通中間層として作り、どこから先を個別Pipelineに任せるか」。この境界をコードの利用実態から定量的に導く。

= 汎用データマート候補

FLOW

分析の流れ — 決定論的に絞り込み、最後だけ意味を足す

STAGE 1
Raw取込
odm_xml_raw / xml_field_mapping

STAGE 2
構造化
pipeline_master / input / output / transform

STAGE 3
グラフ化
graph_node / graph_edge / table_usage_metrics

STAGE 4
連結分析（今回の中心）
join_edge → connected_table_set → stats → parent_child → candidate

STAGE 5
ロジック比較
pipeline_fingerprint / pipeline_cluster

STAGE 6
候補化・意味付け
pipeline_semantic / generic_mart_candidate

STAGE 1-5 は Spark / Databricks SQL のみ。LLM は STAGE 6 の意味付けに限定

01

第N連結 — 何をセットで見るか

第N連結 = N本のJOIN Edgeで連結される N+1 種類のテーブル集合。単なるPATHではなく、同一Pipeline内でJOINにより接続された **Connected Table Set** を対象とする。

A

B

C

D

第1連結 = A + B

第2連結 = A + B + C

第3連結 = A + B + C + D

正規化ルール（必須）

array_sort(table_set) → concat_ws('||', ...) で table_set_key を生成。
CUSTOMER||ACCOUNT||TRANSACTION と TRANSACTION||CUSTOMER||ACCOUNT は必ず同一候補として扱う。

02

維持率で汎用化境界を見つける

※考え方を示す例

第1連結
CUSTOMER + ACCOUNT
1,200 Pipeline
▼ 71%

第2連結 + TRANSACTION
850
▼ 85%

第3連結 + PRODUCT
720
▼ 17% ← 急落

第4連結 + CAMPAIGN
120

読み取り

第3連結までは多数で共通。第4連結で急激に個別化している。

境界候補

CUSTOMER + ACCOUNT + TRANSACTION + PRODUCT までを共通中間層として切り出す。

retention_rate = child / parent parent_support_retention = child / MAX(parent)
max_child_retention が低いほど境界の可能性が高い

03

汎用化候補を判断するシグナル

Pipeline数
何本で同じテーブル群が使われるか

部門数
何部門にまたがるか（件数と独立）

継続性
何年間使われ続けているか

Output種類数
何種類のOutputに再利用されるか

Parent Support Retention
前の連結からどれだけ残るか

Child Retention
足したとき残るか（急落 = 境界）

加工ロジックの類似性
Fingerprintが同一・類似か

下流利用
生成結果がさらに参照されるか

暫定の候補判定（すべてパラメータ化）
pipeline_count >= 5 AND department_count >= 2 AND max_child_retention < 0.5。閾値は業務的な正解ではない。判定から漏れたデータも削除せず保持する。

04

2つの軸を混同しない

Connected Table Set

どのテーブル群が**セット**で使われるか = データ構造の共通性
今回の最優先

Pipeline Fingerprint

そのテーブル群をどう**加工**するか = 加工ロジックの共通性
確度を後から補強

STEP 1 CUSTOMER + ACCOUNT + TRANSACTION が500 Pipelineでセット利用 → 共通中間層候補
STEP 2 JOIN=customer_id / AGG=SUM(amount) / GROUP BY=customer_id, year_month まで一致 → **より強い候補**

Fingerprintは2種類。**Exact** = literal含め完全一致。**Structural** = literalを <LITERAL> 化し構造一致を見る（カラム名・JOINキー・集約・粒度は保持）。SHA-256でHash化。

05

LLMは「構造分析の後の解釈」だけ

Fingerprint → pipeline_cluster → pipeline_semantic → **候補スコアリング**
Hashが違ってても業務的には同じものを作っている場合がある（例：+TRUSTの有無だけ違う「顧客金融資産」）。そのため段階的に束ねてから意味を与える。

LLMへ送るもの

input/output tables、JOIN・集計・粒度の概要、pipeline_count、department_count、active_year_count、downstream数のみ

送らないもの

Raw XML本文、業務データそのもの。20,000件全件も送らない（MAX_LLM_CANDIDATESで上限管理）

根拠のない業務意味は **UNKNOWN**。LLM confidence は定量スコアに混ぜず別管理。endpoint未設定でも構造分析は完走させ、LLM PhaseのみSKIPPEDとして記録する。

06

実装と非交渉のガードレール

P0-3

環境確認 → XML 30件のサンプル観察でマッピング確定 → Raw取込 → 構造化 → テーブル名正規化

P4-7

Data Usage Graph → Fan-in / Fan-out・部門横断・継続性 → Fingerprint → 候補Feature

P8-11

（任意）LLM意味付け → スコアリング → 品質検証 → analysis_run_summary（PASS / WARN / FAIL）

守るべき制約

既存テーブルへの DROP / TRUNCATE / 破壊的OVERWRITE は**禁止**（書き込みは専用Schemaのみ）／XMLにない業務定義を**推測で補完しない**（不明はNULL・UNKNOWN）／20,000×20,000の直積を**作らない**（連結生成はPipeline内で完結）／**MAX_CONNECTION_DEPTH**を必ず設ける（初回は第4連結まで）／Secretを平文で書かない／各フェーズで検証SQLを実行し行数・NULL率・重複・エラーを表示

注意

コード上で頻出していることは、業務定義として正しいことを意味しない

Connected Table Setが頻出していても、JOINキー・JOIN Type・Filter条件・粒度・対象範囲が異なる可能性がある。コード解析結果は「共通データ需要を発見するための証拠」であり、そのまま正解データ定義としない。

進め方

候補から正式な資産まで

コード分析 → Connected Table Setの発見 → 汎用化境界候補の抽出 → Fingerprintによるロジック確認 → **業務定義の確認・統一** → **粒度・更新頻度・SLA・アクセス権の合意** → 正式な汎用データマート